Linked List

A **Linked List** is a linear data structure in which elements are stored in separate memory locations and connected using pointers (or links). Unlike arrays, linked lists do not store elements in contiguous memory locations. Each element, called a **node**, contains two parts: the data and a reference (pointer) to the next node.

🔷 Structure of a Linked List

Each node has:

1. **Data** – The actual value.
2. **Next Pointer** – Address of the next node.

Example representation:

[10 | •] → [20 | •] → [30 | NULL]

Here:

- 10, 20, 30 are data values.
- The last node points to NULL, meaning the list ends there.

🔷 Types of Linked Lists

1️⃣ Singly Linked List

Each node points to the next node only.

A → B → C → NULL

2️⃣ Doubly Linked List

Each node has two pointers:

- One to the next node
- One to the previous node

NULL ← A ⇄ B ⇄ C → NULL

3️⃣ Circular Linked List

The last node points back to the first node.

A → B → C

↑ ↓

← ← ← ←

🔷 Basic Operations

1. **Insertion**
    - At beginning
    - At end
    - At specific position
2. **Deletion**
    - From beginning
    - From end
    - Specific node
3. **Traversal**
    - Visiting each node one by one
4. **Searching**
    - Finding a particular value

🔷 Example in Python (Singly Linked List)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Creating nodes
head = Node(10)
second = Node(20)
third = Node(30)

# Linking nodes
head.next = second
second.next = third
```

🔷 Difference Between Array and Linked List

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Access	Direct (index)	Sequential
Insertion	Slow	Fast

✅ Conclusion

A linked list is a flexible and dynamic data structure that overcomes the fixed-size limitation of arrays. It is widely used in implementing stacks, queues, graphs, and many other advanced data structures. Understanding linked lists is essential for mastering data structures and algorithms.